

Identity en .NET 10

Qué es, cómo funciona y cómo usarlo en aplicaciones Razor Pages

Cuenta · Cookie · Permisos



Identity gestiona el ciclo de vida de las cuentas

Identity no es solo «hacer login».

Gestiona usuarios, contraseñas, perfiles, roles, claims, tokens, confirmación de correo, recuperación de contraseña, autenticación en dos pasos e inicios de sesión externos.

Idea clave: usamos componentes probados y configurables en lugar de reinventar seguridad crítica.

Autenticación y autorización responden preguntas distintas

¿QUIÉN ERES?

Autenticación

Comprueba la identidad y crea un ClaimsPrincipal para la petición.

¿QUÉ PUEDES HACER?

Autorización

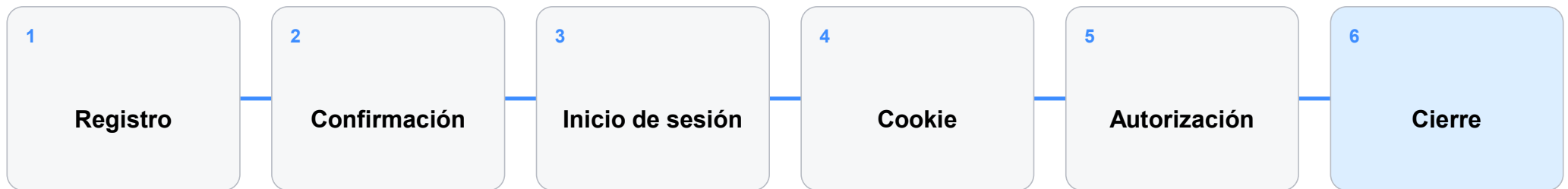
Evalúa usuario autenticado, roles, claims, políticas o el recurso concreto.

¿CÓMO SE GESTIONA?

Identity

Proporciona cuentas, credenciales, tokens, UI y almacenes para apoyar ambos procesos.

El ciclo de una cuenta tiene más pasos que el login



Durante la vida de la cuenta también hay cambios de contraseña, recuperación, bloqueo, datos personales y revocación de sesiones.

Identity conecta la petición HTTP con la base de datos



Las páginas dependen de abstracciones; el proveedor de almacenamiento puede cambiar.

Cuatro piezas concentran casi todo el trabajo diario

ApplicationUser

Representa una cuenta. Puede heredar de IdentityUser y añadir propiedades.

userManager<TUser>

Crea, modifica, busca y valida usuarios, contraseñas, tokens, claims y roles.

SignInManager<TUser>

Inicia y cierra sesión, valida credenciales y coordina cookie, bloqueo y 2FA.

ApplicationDbContext

Hereda de IdentityDbContext y persiste el modelo mediante EF Core.

La plantilla puede crear una aplicación lista para registrarse

Visual Studio

- ASP.NET Core Web App (Razor Pages)
- Framework: .NET 10
- Authentication type: Individual Accounts
- Crear y ejecutar

.NET CLI

```
dotnet new webapp -au Individual -o Peliculas
```

```
cd Peliculas  
dotnet run
```

La plantilla evita configurar a mano el punto de partida.

La plantilla añade servicios, UI, datos y migraciones

Servicios

Registro de Identity, EF Core y Razor Pages en Program.cs.

Datos

ApplicationDbContext, usuario y cadena de conexión.

UI

Área Identity, _LoginPartial y páginas servidas desde una RCL.

Esquema

Migración inicial y tablas AspNetUsers, roles, claims y tokens.

Program.cs registra el almacén y los servicios de Identity

REGISTRO DE SERVICIOS · Program.cs

```
var conexion =  
    builder.Configuration.GetConnectionString("DefaultConnection");  
  
builder.Services.AddDbContext<ApplicationDbContext>(opciones =>  
    opciones.UseSqlite(conexion));  
  
builder.Services  
    .AddDefaultIdentity<ApplicationUser>(opciones =>  
        opciones.SignIn.RequireConfirmedAccount = false)  
    .AddEntityFrameworkStores<ApplicationDbContext>();  
  
builder.Services.AddRazorPages();
```

El middleware reconstruye el usuario

CANALIZACIÓN HTTP · Program.cs

```
var app = builder.Build();

app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();

app.UseAuthentication();
app.UseAuthorization();

app.MapRazorPages();
app.Run();
```



HttpContext.User

Identity usa un modelo relacional preparado para crecer

Tabla	Usuarios	Roles	Relación	Finalidad
AspNetUsers	✓			Cuenta y perfil
AspNetRoles		✓		Roles disponibles
AspNetUserRoles	✓	✓	N:N	Asignación de roles
AspNetUserClaims	✓		1:N	Claims del usuario
AspNetUserLogins	✓		1:N	Proveedores externos
AspNetUserTokens	✓		1:N	Tokens de autenticación

El registro valida antes de guardar y autenticar



Si falla cualquier regla, no se crea la cuenta y se muestran `IdentityResult.Errors`.

UserManager crea la cuenta; SignInManager crea la sesión

REGISTRO · Register.cshtml.cs

```
var usuario = new ApplicationUser
{
    UserName = Input.Email,
    Email = Input.Email
};

IdentityResult resultado =
    await _userManager.CreateAsync(usuario, Input.Password);

if (resultado.Succeeded)
{
    await _signInManager.SignInAsync(usuario, isPersistent: false);
    return LocalRedirect(returnUrl);
}

foreach (IdentityError error in resultado.Errors)
    ModelState.AddModelError(string.Empty, error.Description);
```

La contraseña nunca debe almacenarse en texto plano

Lo que entra

Contraseña escrita por el usuario

«MiClaveSegura!»

Lo que se guarda

Hash de contraseña

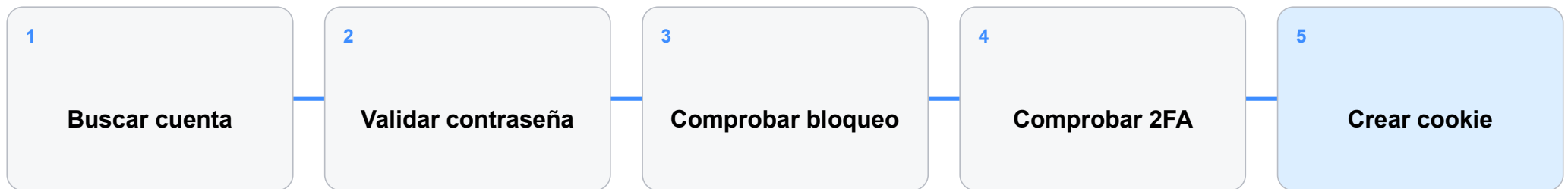
+ salt

+ versión y parámetros

No permite recuperar la contraseña original.

Se compara; no se descifra.

El login decide entre éxito, rechazo, bloqueo o 2FA



El resultado puede ser Succeeded, IsLockedOut, RequiresTwoFactor o Failed.

SignInManager controla el login

LOGIN · Login.cshtml.cs

```
var resultado = await _signInManager.PasswordSignInAsync(  
    Input.Email,  
    Input.Password,  
    Input.RememberMe,  
    lockoutOnFailure: true);  
if (resultado.Succeeded)  
    return LocalRedirect(returnUrl);  
if (resultado.RequiresTwoFactor)  
    return RedirectToPage("./LoginWith2fa");  
if (resultado.IsLockedOut)  
    return RedirectToPage("./Lockout");  
ModelState.AddModelError(string.Empty, "Inicio de sesión incorrecto.");
```


La cookie mantiene la sesión entre peticiones

1 · SERVIDOR

Emite

Tras el login correcto, Identity crea una cookie de autenticación protegida.

2 · NAVEGADOR

Conserva y envía

La cookie viaja automáticamente en peticiones posteriores al sitio.

3 · MIDDLEWARE

Valida

Reconstruye User y sus claims; después se evalúa la autorización.

Para aplicaciones web en navegador, Microsoft recomienda cookies frente a exponer tokens a JavaScript.

Cerrar sesión elimina la cookie de autenticación

LOGOUT · Logout.cshtml.cs

```
public async Task<IActionResult> OnPostAsync(string? returnUrl = null)
{
    await _signInManager.SignOutAsync();

    if (returnUrl is not null)
        return LocalRedirect(returnUrl);

    return RedirectToPage("/Index");
}
```

Regla práctica

- Usar un formulario POST
- Mantener el token antiforgery
- Redirigir a una URL local
- No borrar datos de usuario

[Authorize] protege páginas; [AllowAnonymous] abre excepciones

En el PageModel

```
[Authorize]
public class FavoritasModel : PageModel
{
    public void OnGet() {}
}
```

Mediante convenciones

```
builder.Services.AddRazorPages(opciones =>
{
    opciones.Conventions
        .AuthorizeFolder("/Cuenta");
});
```

La autorización real siempre se comprueba en el servidor.

Roles, claims y políticas expresan permisos con distinto detalle

Mecanismo	Ejemplo	Cuándo usarlo
Rol	Administrador	Agrupación estable de usuarios
Claim	Departamento = Ventas	Dato sobre la identidad
Política	PuedePublicar	Regla reutilizable con requisitos

Los roles requieren registrarlos y asignarlos

ROLES · configuración, asignación y uso

```
builder.Services
    .AddDefaultIdentity<ApplicationUser>()
    .AddRoles<IdentityRole>()
    .AddEntityFrameworkStores<ApplicationDbContext>();

await _roleManager.CreateAsync(new IdentityRole("Administrador"));
await _userManager.AddToRoleAsync(usuario, "Administrador");

[Authorize(Roles = "Administrador")]
public class AdministracionModel : PageModel
{
}
```

Una política convierte claims y reglas en un nombre claro

CLAIMS + POLÍTICA

```
builder.Services.AddAuthorization(opciones =>
{
    opciones.AddPolicy("PuedeModerar", politica =>
        politica.RequireClaim("Permiso", "Moderar"));
});

await _userManager.AddClaimAsync(
    usuario,
    new Claim("Permiso", "Moderar"));

[Authorize(Policy = "PuedeModerar")]
public class ModeracionModel : PageModel
{
}
```

Los datos de negocio se relacionan con el Id del usuario



```
string userId = _userManager.GetUserId(User)!;
```

ApplicationUser permite añadir datos propios al perfil

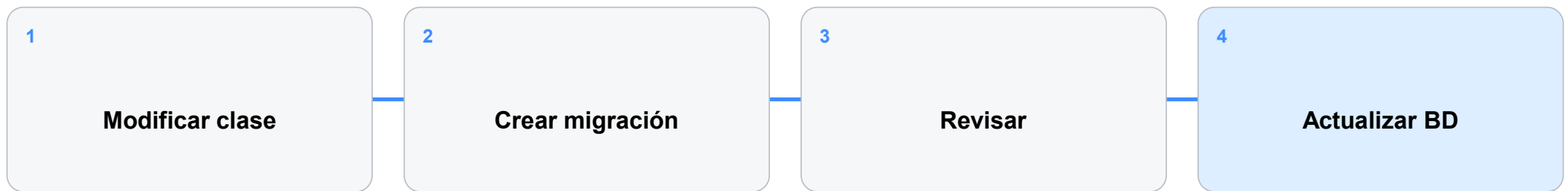
MODELO DE USUARIO PERSONALIZADO

```
public class ApplicationUser : IdentityUser
{
    [PersonalData]
    public string? Nombre { get; set; }

    public DateOnly? FechaNacimiento { get; set; }
}

public class ApplicationDbContext
    : IdentityDbContext<ApplicationUser>
{
    public ApplicationDbContext(
        DbContextOptions<ApplicationDbContext> options)
        : base(options) { }
}
```


Cambiar el modelo exige una migración de EF Core



TERMINAL

```
dotnet ef migrations add AgregarNombreUsuario  
dotnet ef database update
```

IdentityOptions adapta la política de seguridad

CONFIGURACIÓN · contraseñas, bloqueo, usuario y cookie

```
builder.Services.Configure<IdentityOptions>(opciones =>
{
    opciones.Password.RequiredLength = 10;
    opciones.Password.RequireNonAlphanumeric = true;

    opciones.Lockout.MaxFailedAccessAttempts = 5;
    opciones.Lockout.DefaultLockoutTimeSpan =
        TimeSpan.FromMinutes(10);

    opciones.User.RequireUniqueEmail = true;
    opciones.SignIn.RequireConfirmedAccount = false;
});

builder.Services.ConfigureApplicationCookie(opciones =>
{
    opciones.ExpireTimeSpan = TimeSpan.FromMinutes(30);
    opciones.SlidingExpiration = true;
});
```

La confirmación de correo puede ser opcional

Sin confirmación obligatoria

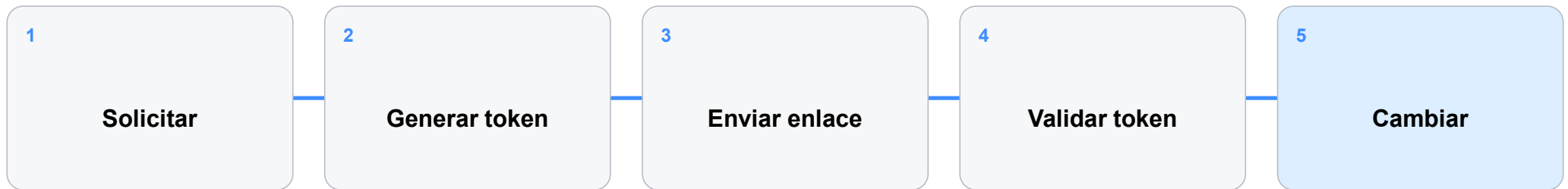
- `RequireConfirmedAccount = false`
- El registro puede iniciar sesión
- Adecuado para prácticas o bajo riesgo
- Debe evitarse confiar en el correo

Con confirmación obligatoria

- `RequireConfirmedAccount = true`
- Se envía un enlace con token
- El login espera confirmación
- Requiere proveedor de correo

La decisión depende del impacto de una cuenta falsa o mal escrita.

Los tokens permiten confirmar y restablecer sin conocer la contraseña



RECUPERACIÓN DE CONTRASEÑA

```
string token =  
    await _userManager.GeneratePasswordResetTokenAsync(usuario);  
  
IdentityResult resultado =  
    await _userManager.ResetPasswordAsync(  
        usuario, token, nuevaPassword);
```

2FA y proveedores externos reducen o trasladan el riesgo

TOTP

Aplicación autenticadora

Añade un código temporal al login. El QR facilita configurar el secreto.

EXTERNO

Google o Microsoft

El proveedor verifica la identidad y devuelve el resultado a la aplicación.

RECUPERACIÓN

Códigos y soporte

Hay que diseñar qué ocurre si se pierde el segundo factor o el proveedor.

La UI predeterminada vive en una Razor Class Library

Usar la UI predeterminada

- Menos archivos propios
- Actualizaciones más sencillas
- Personalización limitada al tema y layout
- Buen punto de partida

Hacer scaffolding

- Elegir solo las páginas necesarias
- Editar Register, Login o Manage
- Revisar los cambios generados
- Aplicar migraciones pendientes

No generes toda la UI si solo necesitas personalizar una página.

Cuatro errores habituales de Identity

Síntoma	Causa probable	Corrección
SignInResult ambiguo	Identity y MVC importados	Usar el nombre completo de Identity
No existen tablas	Migración sin aplicar	dotnet ef database update
Siempre vuelve a Login	Cookie o middleware	Revisar servicios y canalización
No aparece una página	UI servida desde RCL	Scaffold solo si se va a editar

```
Microsoft.AspNetCore.Identity.SignInResult resultado = ...;
```

La seguridad final depende de configuración y disciplina

Transporte y secretos

HTTPS, claves fuera del código, logs sin contraseñas ni tokens.

Cuenta

Bloqueo, confirmación según riesgo, recuperación segura y 2FA.

Autorización

Comprobar en servidor y filtrar datos por el usuario actual.

Operación

Migraciones controladas, actualizaciones, copias y auditoría.

Práctica: películas favoritas por usuario

PARTE 1

Cuentas

Crear Razor Pages con cuentas individuales. Probar registro, login, logout y acceso anónimo.

PARTE 2

Favoritas

Relacionar cada favorita con UserId. Listar, añadir y eliminar solo las del usuario actual.

PARTE 3

Protección

Aplicar [Authorize], validar en servidor y añadir un rol Administrador como ampliación.

Criterio de éxito: dos usuarios nunca pueden modificar las favoritas del otro.

Comprueba si ya dominas las ideas esenciales

- 1 ¿Qué proceso responde «quién eres»?
- 2 ¿Qué servicio coordina el inicio y el cierre de sesión?
- 3 ¿Se guarda la contraseña original en la base de datos?
- 4 ¿Cómo se asocia una película favorita con su propietario?
- 5 ¿Qué consigue el scaffolding de Identity?

Identity aporta la infraestructura; tu aplicación aporta las reglas.

1. Configura servicios y almacenamiento
2. Usa los managers
3. Protege en el servidor
4. Personaliza solo lo necesario

REGISTRO
→ **COOKIE**
→ **PERMISO**